

ENGINEERING / PRODUCT / OPERATIONS

CryptoMacro

Beginner how-to: every file, every important line, how to rebuild

Native Kotlin 2 · Jetpack Compose · Material 3 Adaptive
Hilt · Room · DataStore · EncryptedSharedPreferences
OkHttp HTTPS + Binance WebSocket · Glance widgets
Application ID `com.cryptomacro.app` · minSdk 26 / targetSdk 35
Version 1.0.0 · Built with Gradle 8.11.1 and JDK 17

A. If you have never written an Android app

This section assumes you know how to copy a file and press Run, nothing more. CryptoMacro is a phone app. The user taps icons; the phone runs Kotlin (a language) that asks the internet for prices and draws boxes on the screen with Jetpack Compose (Google's UI toolkit). There is no website inside the app and no company server storing your portfolio.

Words you will see in every file

Word	Plain English
Activity	A window. We have one: MainActivity. Compose draws inside it.
Composable / @Composable	A function that describes UI. When data changes, it redraws.
ViewModel	A box that holds screen data so rotation does not wipe it.
StateFlow / Flow	A pipe of values over time (price went from 70k to 71k).
suspend	This function can wait (network) without freezing the screen.
Hilt / @Inject	Automatic delivery of objects (OkHttp, database) so you do not new them everywhere.
Room / @Entity	SQLite tables with Kotlin classes.
DataStore	A tiny key-value file for settings (dark mode, privacy).
Repository	The only class the UI talks to for data. Hides HTTP and SQL.
apk	The installable file, like a .exe for Android.
minSdk 26	Will not install on Android older than 8.0 (2017).
FLAG_SECURE	Android flag: do not allow screenshots of this window.

How a tap becomes a number on screen (one sentence path)

You tap ETH on Markets. FeaturedChartChips calls `vm.select("eth-usd")`. `MarketsViewModel.selectedId` changes. A coroutine in the `ViewModel` calls `market.candles("eth-usd", D1)`. `MarketRepository` asks `MarketRemoteDataSource`, which GETs Binance klines over OkHttp. The candle list is stored in `candles StateFlow`. `AssetChartCard` collects that Flow and passes the list to `PriceChart`, which draws rectangles on a Canvas. The price label comes from `quotes["eth-usd"]`, which the Websocket or 15-second REST poll keeps updating.

How to read a Kotlin file

- `package ...` is the folder name in Java/Kotlin (`com.cryptomacro.app`).
- `import ...` is 'I need this class from a library.' Skip them on first read.
- The block starting with `/** BEGINNER` is a comment we added for you. Read it first.
- `class Foo : Bar` means `Foo` is a `Bar` with extra behavior.
- `fun name(x: Type): ReturnType { }` is a function. `fun okHttp(): OkHttpClient` builds the HTTP client.
- `val` is a value that does not get reassigned. `var` can change.
- `?.` means 'if this is null, skip.' `?: true` means 'if null, use true.'
- `{ }` after a function name is a lambda: a tiny function passed to someone else.

B. Line-by-line: the files that start the app

B.1 AndroidManifest.xml (the ID card)

Android reads XML before Kotlin. uses-permission INTERNET lets OkHttp call Binance. USE_BIOMETRIC lets the lock screen ask for a fingerprint. application allowBackup=false means Google Backup will not copy your holdings. usesCleartextTraffic=false forbids http://. The launcher intent-filter is why the Bitcoin icon appears on the home screen. Debug builds still use this file; Gradle adds .debug to the package name automatically.

B.2 CryptoMacroApp.kt (the process)

```
@HiltAndroidApp                                // start Hilt, otherwise @Inject crashes
class CryptoMacroApp : Application(),            // one instance per process
    Configuration.Provider {                    // WorkManager asks us how to build workers
    @Inject lateinit var workerFactory          // Hilt fills this after super.onCreate
    override val workManagerConfiguration      // 'use Hilt to construct PriceSyncWorker'
    override fun onCreate() {
        super.onCreate()                        // required Android setup
        PriceSyncWorker.enqueue(this)          // schedule 15-min widget refresh
    }
}
```

B.3 AppModule.kt (how objects are created)

```
fun okHttp(): OkHttpClient = OkHttpClient.Builder()
    .connectTimeout(8, SECONDS)                // give up connecting
    .readTimeout(12, SECONDS)                  // give up waiting for bytes
    .followRedirects(false)                    // no http surprise redirects
    .addInterceptor { chain ->
        check(url.scheme == "https")          // throw if someone passes http://
        chain.proceed(request)                // send
    }.build()

fun db(context): AppDatabase =
    Room.databaseBuilder(..., "cryptomacro.db") // file in /data/data/.../databases
        .fallbackToDestructiveMigration()      // schema mismatch = wipe (we keep version 1)
        .build()
```

B.4 MainActivity.kt (the only screen host)

onCreate is 'the window is born.' We install the splash, call super.onCreate, then immediately FLAG_SECURE. setContent {} is Compose. collectAsStateWithLifecycle turns a Flow into a value that redraws the UI. If settings is still null we draw nothing (no flash of the portfolio). needsLock = biometric enabled AND not yet unlocked. onNewIntent handles a shortcut while the activity already exists (launchMode=singleTop). promptBiometric shows the system sheet; only onAuthenticationSucceeded sets unlocked = true.

```
window.addFlags(FLAG_SECURE)                  // line: block recents screenshot
val settings by preferences.settings.collect(...) // wait for DataStore
if (ready == null) return@setContent          // still loading
if (needsLock) LockScreen(...) else AppRoot(...)
```

B.5 AppRoot.kt (four tabs)

dest is which tab is selected (remember { } keeps it across redraws). NavigationSuiteScaffold draws the bar. classifyLayout(width, height): width<600 Compact; else if width<840 OR height<500 Medium; else Expanded. The height check is why a Pixel in landscape does not get three skinny columns. onPreviewKeyEvent only reacts when Ctrl is held so amount fields can type 1, 2, 3, 4.

C. Line-by-line: prices and portfolio

C.1 HttpJson.get

```
suspend fun get(url: String): String = withContext(Dispatchers.IO) {
    assertHttps(url) // must start with https://
    val ua = if (yahoo) ChromeUA else "CryptoMacro/1.0"
    val req = Request.Builder().url(url).header("User-Agent", ua).build()
    client.newCall(req).execute().use { res ->
        if (!res.isSuccessful) error("HTTP ${res.code}")
        res.body?.string() ?: error("Empty body")
    }
}
```

withContext(IO) means 'run this on a background thread.' execute() is a blocking HTTP call. use { } closes the response even if we throw. Yahoo needs a Chrome User-Agent or it returns 401.

C.2 BinanceWebSocket.ticks

callbackFlow turns socket callbacks into a Flow. We join symbols as btcusdt@miniTicker/ethusdt@miniTicker. The Request URL is https://stream.binance.com:9443/stream?streams=... (not wss://). onMessage parses JSON data.s (symbol) and data.c (close price). Invalid symbols are dropped. awaitClose { socket.close } runs when the UI leaves and nobody is collecting the Flow.

C.3 MarketRepository init loops

```
scope.launch { while (true) { refreshTopStocksLive(); delay(10_000) } } // Yahoo spark
scope.launch { while (true) { refreshTopCryptos(); delay(60*60_000) } } // CoinGecko rank
scope.launch { while (true) { delay(15_000); refreshCryptoLivePrices() } } // Binance 24hr
```

These infinite coroutines live in a SupervisorJob: one failure does not cancel the others. refreshTopCryptos() fills extraAssets so cg-dogecoin can be resolved later by the chart and portfolio.

C.4 Adding a portfolio transaction

```
// UI: user tapped Save
val a = Formatters.parseDecimal(amount) // "0,5" -> 0.5
vm.addTransaction(selectedId, BUY, a, price)

// ViewModel
val asset = market.resolve(assetId) ?: return
registry.ensureTracked(asset) // Room custom_assets so restart still knows the name
portfolio.addTransaction(...) // SQL insert
market.refreshQuotes(listOf(asset)) // total is not $0 while waiting for the 15s poll
```

summary() does valueUsd = holding.amount * livePrice and sums every row into Portfolio total. If the asset id cannot be resolved we still show a placeholder so a holding never vanishes.

C.5 PriceChart drawing

Canvas { } gives a DrawScope. We map each candle to an x (time) and y (price) pixel. Bull candles are filled teal; bear candles are red. SMA is the average of the last 20 closes. detectTransformGestures: zoom is multiplied into scale; pan changes offsetX. Long-press computes which candle index sits under the finger and calls onCrosshair.

D. Every Kotlin file (what to open first)

Open this	What a beginner should learn from it
CryptoMacroApp.kt	Process start, Hilt, WorkManager hook
di/AppModule.kt	How OkHttp and Room are constructed once
MainActivity.kt	Splash, FLAG_SECURE, biometric, setContent
ui/navigation/AppRoot.kt	Four tabs and phone vs tablet layout
ui/markets/MarketsScreens.kt	Chips, Add sheet, list, chart card
ui/markets/MarketsViewModel.kt	select, pinChart, listed combine
ui/chart/PriceChart.kt	Canvas, gestures, loading placeholder
data/remote/HttpJson.kt	HTTPS GET cache
data/remote/BinanceWebSocket.kt	Push prices
data/remote/MarketRemoteDataSource.kt	Every public API adapter
data/repository/MarketRepository.kt	The live-price brain
data/repository/PortfolioRepository.kt	Holdings math
data/local/Entities.kt + Daos.kt	SQL tables
data/local/PreferencesRepository.kt	Settings keys
data/local/SecureStore.kt	Encrypted API keys
ui/portfolio/*	Total, Add transaction sheet
ui/learn/*	Three tabs of education
ui/settings/*	Debug-gated secrets
widget/ and tile/	Home screen and shade
worker/PriceSyncWorker.kt	Background refresh
domain/model/Assets.kt	Built-in coins and timeframes

Each of those files now starts with a BEGINNER comment block. Read the comment, then the first function, then the imports last.

1. Purpose of this document

This guide explains how the CryptoMacro Android application was created: the original web dashboard it replaced, the native stack that was chosen, every major module, the live-data pipeline, local security model, adaptive layouts for phones and tablets, and the exact Gradle / ADB commands used to compile and install it. It describes the as-built product, not the earlier web prototype or features that were later removed (QR scan, deposit/withdraw, JSON export, address watch).

Field	Value
Product name	CryptoMacro
Package / applicationId	com.cryptomacro.app (debug suffix .debug)

Field	Value
Version	1.0.0 (versionCode 1)
Language	Kotlin 2.0.21 (100% Kotlin, coroutines + Flow)
UI	Jetpack Compose + Material 3 + NavigationSuiteScaffold
Min / compile / target SDK	26 / 35 / 35
Build system	Gradle 8.11.1, Android Gradle Plugin 8.7.3, KSP
JDK	17 (Microsoft OpenJDK 17 used on the authoring machine)
Source of truth spec	docs/SPEC.md (product requirements)

2. Origin story — from web dashboard to native app

CryptoMacro began as a React + Vite + TypeScript dashboard (CryptoDashboardApp) with Playwright tests and a GitHub Pages workflow. The product goal was retail-friendly market context: live crypto prices, S&P 500 and gold as macro benchmarks, Fear & Greed, Bitcoin halving, and an educational Shemitah 7-year overlay. Users asked for a real Android app that would run on any commercial phone and tablet, with local-only portfolio data and no custody of seeds or private keys.

What happened in the port

- The web tree (package.json, Vite, Playwright, GH Pages, src/*.tsx) was deleted once the Compose app compiled.
- The product spec was kept as docs/SPEC.md and implemented in Kotlin rather than wrapping a WebView.
- Features that implied custody or extra permissions were stripped after review: Deposit/Withdraw, Scan QR, camera, NFC, Export JSON, Watch address.
- Play Store / release builds hide developer Feed status and exchange API-key fields (gated with BuildConfig.DEBUG).
- The app was developed and regression-tested on a physical Pixel 10 Pro XL (USB debugging), not an emulator (host disk/CPU were too tight).

3. How to recreate the project from zero

3.1 Machine prerequisites

- Windows, macOS, or Linux with at least 8 GB RAM (16 GB preferred).
- JDK 17. On Windows the project was built with Microsoft OpenJDK 17 at C:\Program Files\Microsoft\jdk-17.0.20.8-hotspot. Set JAVA_HOME.
- Android SDK with platform android-35, build-tools 34 or 35, and platform-tools (adb). SDK root can live off C: (this project used D:\Android\Sdk).
- Android Studio Ladybug / 2024.2+ is optional but convenient. Gradle Wrapper does not require a global Gradle install.
- A hardware device with USB debugging (min API 26). Emulators work if the host can spare RAM.

3.2 Create a blank Android app, then drop in this architecture

If you are inventing the app again rather than cloning this folder: File → New → Empty Compose Activity in Android Studio, set applicationId to com.cryptomacro.app, minSdk 26, compileSdk 35, enable Compose and BuildConfig.

Then copy the package layout in section 5 and the version catalog in section 4. The empty-activity template already gives you MainActivity + Theme; replace them with Hilt + NavigationSuiteScaffold as below.

3.3 local.properties

```
sdk.dir=D:\\Android\\Sdk
# or on macOS/Linux:
# sdk.dir=/Users/<you>/Library/Android/sdk
```

3.4 First compile and install

```
# Windows
set JAVA_HOME=C:\\Program Files\\Microsoft\\jdk-17.0.20.8-hotspot
set ANDROID_SDK_ROOT=D:\\Android\\Sdk
gradlew.bat :app:assembleDebug --no-daemon

adb install -r -t app\\build\\outputs\\apk\\debug\\app-debug.apk
adb shell am force-stop com.cryptomacro.app.debug
adb shell am start -n com.cryptomacro.app.debug/com.cryptomacro.app.MainActivity

# macOS / Linux
./gradlew :app:assembleDebug
adb install -r -t app/build/outputs/apk/debug/app-debug.apk
```

Debug builds append .debug to the application ID so a Play Store package and a sideload can coexist. The launcher activity is always com.cryptomacro.app.MainActivity.

4. Toolchain, plugins, and libraries (exact versions)

Versions live in gradle/libs.versions.toml (version catalog). Root build.gradle.kts only applies plugins as false; the :app module consumes them. This is the inventory that actually compiled.

Component	Version / artifact	Why it is here
Gradle Wrapper	8.11.1	Declared in gradle/wrapper/gradle-wrapper.properties
Android Gradle Plugin	8.7.3	com.android.application
Kotlin	2.0.21	android + compose + serialization plugins
KSP	2.0.21-1.0.28	Hilt and Room annotation processing
Compose BOM	2024.12.01	Pins Compose UI / Material3
Material 3 Adaptive	1.0.0 + nav suite 1.3.1	NavigationSuiteScaffold, list-detail
Hilt	2.52	App-wide DI; HiltWorkerFactory for WorkManager
Room	2.6.1	Holdings, transactions, candle/quote cache, custom assets
OkHttp	4.12.0	HTTPS REST + WebSocket (no Retrofit/Ktor)
kotlinx.serialization	1.7.3	JSON parse of public APIs
DataStore	1.1.1	Theme, fiat, biometric, privacy, pinned charts
security-crypto	1.0.0	EncryptedSharedPreferences AES-256-GCM
Biometric	1.1.0	BiometricPrompt + device credential

Component	Version / artifact	Why it is here
WorkManager	2.10.0	15-minute widget price sync
Glance	1.1.1	Home-screen portfolio + ticker widgets
WindowManager	1.3.0	FoldingFeature / tabletop posture
SplashScreen	1.0.1	Android 12+ splash with Bitcoin coin art

CameraX and ML Kit barcode libraries remain in the version catalog from an earlier QR experiment but are not on the app classpath anymore. The QR screen, CAMERA permission, and scan shortcut were removed.

5. Repository layout

```

CryptoDashboardApp/
  settings.gradle.kts      root name CryptoMacro, include :app
  build.gradle.kts         plugin aliases only
  gradle/libs.versions.toml version catalog
  gradle/wrapper/          Gradle 8.11.1
  docs/SPEC.md             original product spec
  docs/CryptoMacro_Technical_Build_Guide.pdf this document
  app/
    build.gradle.kts       application module
    proguard-rules.pro     release minify (keep serializers, Glance, tiles)
    src/main/
      AndroidManifest.xml
      assets/spx.json      offline S&P snapshot fallback
      java/com/cryptomacro/app/
        MainActivity.kt    splash, FLAG_SECURE, biometric gate
        CryptoMacroApp.kt @HiltAndroidApp + WorkManager
        di/                OkHttp + Room
        data/local|remote|repository/
        domain/model|util/
        ui/markets|portfolio|learn|settings|chart|navigation|lock
        widget/ tile/ worker/
      res/ mipmaps, network_security_config, backup_rules, shortcuts

```

6. Runtime architecture

The app is a single-module MVVM Compose client. There is no backend. Market data is pulled from public HTTPS APIs; portfolio data never leaves the phone. Unidirectional data: repositories expose StateFlow, ViewModels map it for the UI, composables collect with lifecycle awareness.

```

UI (Compose + NavigationSuiteScaffold)
  MarketsViewModel / PortfolioViewModel / LearnViewModel / SettingsViewModel
  MarketRepository (quotes, ticks, candles, top lists)
  PortfolioRepository (Room holdings + transactions + DCA math)
  AssetRegistry (CoreAssets + Room custom/pinned)
  PreferencesRepository (DataStore)
  SecureStore (EncryptedSharedPreferences)
    OkHttp (TLS only, no redirects) → Binance / CoinGecko / Yahoo / mempool / Frankfurter
    Room cryptomacro.db
    DataStore cryptomacro_settings
    EncryptedSharedPreferences cryptomacro_secure

```

6.1 Dependency injection

Hilt @HiltAndroidApp on CryptoMacroApp. AppModule provides a singleton OkHttpClient (8s connect, 12s read, followRedirects false, interceptor that rejects any non-https scheme) and a Room database named cryptomacro.db

with `fallbackToDestructiveMigration`. ViewModels use `@HiltViewModel`. Glance widgets and QS tiles cannot use constructor injection; they reach repositories through `WidgetEntryPoint` (an `@EntryPoint` on `SingletonComponent`).

6.2 Application class and process start

`CryptoMacroApp` implements `Configuration.Provider` so `WorkManager` uses `HiltWorkerFactory`. `onCreate` enqueues `PriceSyncWorker` as unique periodic work named `price-sync`, 15 minutes, `ExistingPeriodicWorkPolicy.KEEP`. `MainActivity` installs the splash, enables edge-to-edge (dark transparent system bars), sets `FLAG_SECURE` immediately, then waits for `DataStore` settings before composing `AppRoot` so biometric lock cannot flash holdings.

7. Features as they actually shipped

7.1 Markets

- Pinned chart chips: BTC, ETH, SOL, SPX plus user pins (crypto from top 200 or stocks from top 50). Add chip opens a searchable sheet with `Crypto / Stocks` tabs.
- Custom Compose Canvas chart: candlestick / line / area, pinch zoom, pan, long-press OHLC crosshair, SMA 20, EMA 12, volume, optional Shemitah bands.
- Timeframes wrap on small screens: 1H, 4H, 24H, 7D, 30D, 90D, 1Y, ALL.
- Lists: Top 200 crypto (CoinGecko markets, two pages of 100), Top 50 US stocks (Yahoo spark batches of 20), DeFi / L1-L2 / Memes from `CoreAssets`, Watch = defaults + pins.
- Live pill: green Live when Binance combined miniTicker WebSocket is up; otherwise REST.
- Refresh icon reloads ranking and quotes. The old top-right + (Binance pair catalog) was removed; Add lives beside the chips.

7.2 Portfolio

- Local holdings and a transaction ledger (Buy, Sell, Transfer in/out, Staking).
- Add transaction sheet searches the live top-200 list, auto-fills USD price, accepts comma or dot decimals.
- Portfolio total = $\text{sum}(\text{amount} \times \text{live USD price})$, converted to EUR via Frankfurter (ECB) with CoinGecko fallback.
- Allocation donut, per-asset cards, Remove holding, DCA calculator versus weekly/monthly BTC schedules.
- No deposit, withdraw, QR, export JSON, or chain-address watch in the shipped UI.

7.3 Learn

- Sentiment: Fear & Greed gauge (alternative.me), Bitcoin halving countdown (mempool.space tip height), mempool fee ladder.
- Cycles: Shemitah phase, historical averages, guidance windows, timeline of stress markers, disclaimer.
- Lessons: in-app articles on risk, custody, 2FA, leverage (EducationCatalog).
- On wide tablets, Lessons splits into picker + article panes.

7.4 Settings / Security

- Theme SYSTEM / DARK / LIGHT, fiat USD / EUR.
- Biometric app lock (BIOMETRIC_STRONG or DEVICE_CREDENTIAL). Lock now returns to LockScreen.
- Privacy shield: FLAG_SECURE (default ON) blocks screenshots, recents thumbnails, and screen share.
- Chart overlay toggles: SMA, EMA, volume, Shemitah bands.
- Debug-only: feed source blurb and encrypted read-only API key forms for Binance / Coinbase / Kraken. Hidden when `BuildConfig.DEBUG` is false.

8. Market data pipeline (invented integration layer)

There is no single vendor SDK. MarketRemoteDataSource + HttpJson + BinanceWebSocket are the custom client. HttpJson caches GET bodies in memory with TTL, coalesces in-flight requests, sends a browser User-Agent to Yahoo (required; Yahoo v7 quote returns 401) and CryptoMacro/1.0 otherwise, and refuses non-HTTPS URLs.

Source	Endpoint / protocol	Used for	Cadence
Binance	wss via https://stream.binance.com:9443 combined miniTicker	Live crypto last price	push; reconnect backoff to 30s
Binance REST	/api/v3/ticker/24hr, /klines, /exchangeInfo	24h change, candles, USDT catalog	15s live overlay; candles on demand
CoinGecko	/coins/markets?per_page=100&page=1..2	Top 200 ranking, caps, names, ids	hourly
Yahoo Finance	/v8/finance/chart and /v7/finance/spark	SPX/gold candles; top 50 stock quotes	spark every 10s, max 20 symbols
CoinLore	/api/global/ + tickers	Total cap, BTC.D fallback	overview 60s
alternative.me	/fng/	Fear & Greed value + label	overview 60s
mempool.space	tip/height, /v1/fees/recommended	Halving math, sat/vB ladder	on Learn open
Frankfurter	/latest?from=USD&to=EUR	Official EUR FX	portfolio open
CryptoCompare	histominute/hour/day	Candle failover if Binance fails	on demand
Local asset	assets/spx.json	SPX if Yahoo is blocked	on demand

8.1 Ranking versus last price

CoinGecko ranking is expensive and rate-limited, so it runs hourly. Between ranking refreshes, crypto last prices come from the Binance 24hr ticker (all symbols, one call, every 15s) mapped by binanceSymbol, plus the WebSocket. Stocks cannot use Yahoo v7 quote (HTTP 401); spark with a Chrome UA and batches of 20 symbols is the working path. If a spark batch fails, previous quotes are kept so the Top 50 list does not collapse.

8.2 WebSocket details (easy to get wrong)

- OkHttp HttpUrl only accepts http/https. The socket URL must be https://stream.binance.com:9443/... ; OkHttp upgrades it to WSS. A wss:// Request.Builder URL throws and the Live pill stays on REST.
- Symbols are allowlisted with `^[A-Z0-9]{4,20}$` and capped at 200 streams.
- The HTTPS interceptor must allow https only; wss is not seen by interceptors after the upgrade.
- Ticks update MarketRepository._quotes and _ticks; the chart still uses REST/cached klines until the next candle fetch.

8.3 Candle engine

PriceChart is a custom Compose Canvas (not MPAndroidChart / TradingView). It draws candles or an area path, volume histogram, SMA/EMA polylines, and Shemitah year bands. Gestures: detectTransformGestures for pinch/pan, long-press for crosshair. If fewer than two candles exist it shows Loading chart... instead of collapsing the card. Zoom resets when the series identity changes. Synthetic sine-wave candles are a last-resort offline illustration and are labelled as such.

9. Local persistence and security model

9.1 Room (unencrypted SQLite, backups off)

Entities: candle_cache, quote_cache, custom_assets, holdings, transactions, watch_addresses. Version 1, exportSchema false, destructive migration. Watch-address tables remain from the removed feature so we do not wipe user holdings with a schema bump. custom_assets stores pinned/non-core coins so portfolio rows and chart pins survive process death. Stock rows use id prefix stock- and are reconstituted with yahooSymbol on read.

9.2 DataStore preferences

File cryptomacro_settings. Keys: theme, biometric, privacy (default true), fiat, favorite, sma/ema/volume/shemitah overlays, pinned_charts (comma-separated asset ids, max 24 extras).

9.3 EncryptedSharedPreferences

SecureStore creates MasterKeys AES256-GCM and EncryptedSharedPreferences named cryptomacro_secure (key AES256_SIV, value AES256_GCM). There is no plaintext fallback. If Keystore creation fails, the store is null and API-key save is a no-op (fail closed). Only debug UI writes keys.

9.4 Network and OS policy

- network_security_config: cleartextTrafficPermitted false, system CAs only (no user CAs).
- android:usesCleartextTraffic=false, android:allowBackup=false, cloud and device-transfer extraction rules exclude sharedpref and database.
- Permissions actually used: INTERNET, ACCESS_NETWORK_STATE, USE_BIOMETRIC, USE_FINGERPRINT. WorkManager merges RECEIVE_BOOT_COMPLETED, WAKE_LOCK, FOREGROUND_SERVICE.
- FLAG_SECURE default on before first frame. Privacy tile and Settings toggle it. Portfolio Glance widget hides balances when privacy or biometric is on.
- Release ProGuard: minify + shrink resources; keep kotlin serialization for domain models, Glance widgets, tiles, workers — not the entire application package.

10. Adaptive UI for phones, foldables, tablets (2026 form factors)

NavigationSuiteScaffold picks bottom bar vs rail automatically. Content layout is classified in AppRoot from Configuration.screenWidthDp and screenHeightDp so a landscape phone does not get a cramped 3-pane:

Kind	Rule	Markets layout
Compact	width < 600 dp	Single column: chips + chart + list. Detail is a full-screen pane with back.
Medium	width < 840 or height < 500	List 42% + chart/metrics 58%. Landscape phones stay here.
Expanded	width ≥ 840 and height ≥ 500	List 28% + chart 44% + holdings/DCA context 28%.
Tabletop fold	FoldingFeature HALF_OPENED + HORIZONTAL	Chart top half, portfolio bottom half.

safeDrawing insets (top + horizontal) keep titles out of the status bar / punch-hole. Pixel 10 Pro XL measured statusBars 161 px and navigationBars 59 px. windowSoftInputMode=adjustResize plus imePadding on sheets. Portfolio and Settings cap readable width at 720 dp on large tablets. Activity is resizeable, supports all screen sizes, cutout shortEdges, and configChanges include density/screenSize so free-form Android 15+ windows relay layout in place.

Keyboard: Ctrl+1..4 switches tabs, Ctrl+F Markets, Ctrl+R refresh. Digits without Ctrl are left for amount fields (an early bug stole 1-4 from the transaction form).

11. Widgets, tiles, shortcuts

Surface	Class	Behavior
Price ticker widget	PriceTickerWidget / Receiver	Favorite asset last price (public market data).
Portfolio widget	PortfolioWidget / Receiver	Total + 24h; shows Hidden if privacy or biometric on.
QS price tile	PriceTileService	BTC / ETH compact USD; job cancelled in onStopListening.
QS privacy tile	PrivacyTileService	Toggles FLAG_SECURE preference.
Launcher shortcut	res/xml/shortcuts.xml	Add buy transaction → MainActivity ACTION_ADD_TRANSACTION. No targetPackage so debug suffix works. Scan shortcut removed.

12. Core domain model

CoreAssets in domain/model/Assets.kt is the curated set: L1 (BTC ETH SOL ADA AVAX DOT NEAR), L2 (ARB OP MATIC), DeFi (LINK UNI AAVE), memes (DOGE SHIB PEPE), stables, SPX, gold, TOTAL/TOTAL2/TOTAL3/BTC.D, ETHBTC pairs, Fear & Greed. TopStocks.kt lists 50 large-cap US tickers (AAPL ... UBER) mapped to ids stock-{symbol}. CoinGecko rows that match a CoreAssets.coingeckold reuse the core id (so Bitcoin stays btc-usd). Everything else is cg-{coingeckold}.

13. Scripts and commands actually used

There is no Node prefetch script anymore (the old scripts/prefetch-spx.mjs died with the web app). Operational scripts are Gradle + ADB + a Python PDF generator.

Command	Role
gradlew.bat :app:assembleDebug --no-daemon	Compile and package the sideload APK (used throughout development).
gradlew.bat :app:assembleRelease	Minified Play-style APK (API key UI compiled out of the Settings tree via BuildConfig).
adb -s <serial> install -r -t app-debug.apk	Replace debug install; -t allows test-only if needed.
adb shell am force-stop ... ; am start -n .../MainActivity	Cold start after install (otherwise Android may only bring an old task forward).
adb shell am start -W -n ...	Wait until first frame; TotalTime is launch cost.
adb shell uiautomator dump /sdcard/uidump.xml	Regression: read Compose text nodes (FLAG_SECURE blocks screencap).
adb logcat --pid=<pid>	Process-local logs; crash buffer is logcat -b crash.
python docs/generate_build_guide.py	Regenerates this PDF with reportlab.

Authoring-machine environment (reference)

```
OS: Windows
JAVA_HOME=C:\Program Files\Microsoft\jdk-17.0.20.8-hotspot
ANDROID_SDK_ROOT=D:\Android\Sdk
Device: Pixel 10 Pro XL product=mustang serial=57091FDCQ003U4
Physical size 1080x2404 density 390
```

14. How the UI was tested

Instrumentation tests were not the primary loop (no emulator budget). QA was: assembleDebug, adb install, uiautomator dump of every destination, tap chips/filters/sheets, then logcat for FATAL. Confirmed on device:

- Markets Live pill, BTC/ETH/SOL/SPX/Add chips, live Bitcoin and Ethereum prices, Fear & Greed and global cap line.
- Add-chart sheet: Crypto tab (BTC/ETH pinned, USDT, BNB), Stocks tab (AAPL, NVDA, MSFT, GOOGL with spark prices).
- Top 50 stocks list after scroll (MSFT through Visa with 24h deltas).
- Portfolio total, Add transaction sheet listing top-200 with live USD.
- Learn Sentiment (F&G, halving height, mempool fees), Cycles, Lessons.
- Settings appearance, privacy, biometrics, Lock now. Debug API-key cards only after scrolling (debug APK).
- No process crash in AndroidRuntime for com.cryptomacro.app.debug during the pass.

Privacy shield is on by default, so adb exec-out screencap is black. Use uiautomator dump, not screenshots, for automation.

15. Play Store / release checklist

- assembleRelease with minify enabled. Confirm Settings has no Feed or API key fields (BuildConfig.DEBUG false).
- applicationId com.cryptomacro.app (no .debug suffix).
- Privacy policy: local-only portfolio, public market APIs, no account, no seed storage.
- Data safety form: no shared data, no encryption-in-transit of user PII (there is none); FLAG_SECURE and backup exclusion.
- targetSdk 35 satisfies current Play target requirements; bump compile/target when Play mandates 36.
- Widgets must not leak balances if the user enabled privacy — already implemented.
- Content rating: informational finance tool; include the in-app disclaimer string.

16. Invented pieces (not third-party products)

These are original to this codebase rather than drop-in libraries:

- HttpJson — TTL memory cache, in-flight coalescing, Yahoo UA switch, HTTPS assert.
- BinanceWebSocket — combined stream builder, symbol sanitizer, exponential reconnect.
- MarketRemoteDataSource — failover ladder Binance → CryptoCompare → CoinGecko OHLC; aggregate TOTAL/BTC.D reconstruction from BTC/ETH klines + global snapshot; Yahoo spark stock quotes; synthetic seed series.
- PriceChart — Compose Canvas financial chart with SMA/EMA/volume/Shemitah overlays and gestures.
- ShemitahData / EducationCatalog — educational cycle overlay and lesson copy.
- classifyLayout — width+height heuristic so landscape phones stay two-pane.
- Pinned chart ids in DataStore + custom_assets so user charts survive restarts.
- Portfolio DCA engine comparing actual buys to weekly/monthly equal-cash schedules on candle history.

17. Step-by-step: implement the app in order

If a new engineer rebuilds from a blank Compose project, this order avoids circular dependencies:

- 1. Gradle catalog, Hilt application class, empty MainActivity with splash + edge-to-edge.
- 2. Theme (dark default #0B0E14, bull #00C087, bear #F6465D, gold #F59E0B) and AppCard/DeltaText/MoneyText.
- 3. AppModule OkHttp TLS interceptor + Room database + entities/DAOs.
- 4. HttpJson + MarketRemoteDataSource + BinanceWebSocket + MarketRepository (quotes StateFlow).
- 5. AppRoot + NavigationSuiteScaffold with four destinations and classifyLayout.
- 6. Markets list + PriceChart + timeframe chips; then top-200 / top-50 feeds.
- 7. Featured chips + Add-chart sheet + DataStore pinned ids + AssetRegistry.ensureTracked.
- 8. Portfolio Room holdings/transactions, summary combine with live quotes, Add transaction sheet.
- 9. Learn tabs (sentiment APIs, Shemitah, lessons).
- 10. Settings, biometric lock, FLAG_SECURE, debug-only API keys.
- 11. Glance widgets, QS tiles, WorkManager, shortcuts.
- 12. ProGuard, backup exclusion, network security config, release BuildConfig gate.
- 13. Device QA with adb install + uiautomator dump on compact; resize on a tablet or wm size for expanded.

18. Key source files (quick map)

File	Responsibility
MainActivity.kt	Splash, FLAG_SECURE, biometric gate, folding tracker, shortcut intent
ui/navigation/AppRoot.kt	Four tabs, keyboard, compact/medium/expanded/tabletop
ui/markets/MarketsScreens.kt	Chips, chart card, lists, Add-chart sheet
ui/chart/PriceChart.kt	Canvas renderer
data/remote/MarketRemoteDataSource.kt	All HTTP market adapters
data/remote/BinanceWebSocket.kt	Live ticks
data/repository/MarketRepository.kt	Orchestrates ranking, polling, WS, cache
data/repository/PortfolioRepository.kt	Holdings math, DCA, ledger
data/local/SecureStore.kt	Keystore-backed API secrets
di/AppModule.kt	OkHttp + Room
widget/*.kt, tile/*.kt, worker/PriceSyncWorker.kt	OS surfaces

19. Disclaimer and product boundaries

CryptoMacro is a local market-context and journaling tool. It is not financial advice. It does not place orders, custody assets, store seed phrases, or operate a backend account. Live prices can be delayed, incomplete, or unavailable. Shemitah statistics are historical educational overlays; they are not a trading signal. Exchange API fields, when visible in debug, must be withdraw-disabled read-only keys.

20. Document history

Date	Event
2026-08	Native Compose app replaces React/Vite dashboard; spec retained as docs/SPEC.md.
2026-08	Security pass: HTTPS-only OkHttp, encrypted prefs, FLAG_SECURE, permission trim, WS URL fix.
2026-08	Portfolio top-200 picker; Markets Add chip for crypto/stocks; QR/export/watch removed.
2026-08-20	On-device regression on Pixel 10 Pro XL; this build guide generated.

End of guide. Rebuild this file with: `python docs/generate_build_guide.py`